SYSTEM DESIGN INTERVIEW | FREE PRINTABLE GUIDE

30 questions and answers

15 interactive design prompts with answer outlines and worked walkthroughs, plus 15 system design fundamentals with direct answers and links to deeper guides. Free to download, with no account or email required.

Use each outline as a starting point. In an interview, clarify requirements and scale first; a sound design depends on the constraints you agree on.

Design a URL Shortener

Answer outline. Generate a unique key, store its mapping to the long URL durably, and keep redirects fast with a cache for popular keys. Record click analytics asynchronously so tracking does not slow the redirect path.

Practice focus. Estimate a read-heavy workload, then decide what to cache and how to count clicks without slowing redirects.

Read the full worked answer

Design an API Rate Limiter

Answer outline. A token bucket handles bursts while enforcing an average refill rate. A strict global limit needs shared state across servers; local counters are more available but can temporarily exceed the intended limit.

Practice focus. Compare counter and token-bucket designs, then decide how multiple servers share limits.

Read the full worked answer

Design a News Feed (Twitter Timeline)

Answer outline. Build most feeds by writing new posts into followers' timelines, then fetch posts on read for very large publishers. A hybrid avoids making celebrity posts fan out to millions of inboxes while keeping ordinary reads fast.

Practice focus. Choose between write fan-out and read fan-out when a few accounts have millions of followers.

Read the full worked answer

Design an Uber-Style Ride-Sharing System

Answer outline. Use a short-lived spatial index to find fresh, available drivers near a pickup, then rank a small candidate set by road ETA. Reserve the driver and ride with a conditional, versioned state transition before sending an offer; leases and idempotent replies make retries safe.

Practice focus. Design Uber-style dispatch: match nearby drivers quickly without stale locations or double assignments, then isolate a stadium hotspot.

Read the full worked answer

Design a Payment System

Answer outline. Give each payment attempt a durable idempotency key and move it through an explicit state machine. Retries can then return the existing result, while provider events are deduplicated and reconciled instead of assuming exactly-once delivery.

Practice focus. Make retries safe when a payment succeeds but the caller never receives the response.

Read the full worked answer

Design a Distributed Job Queue

Answer outline. Assume at-least-once delivery: acknowledge work only after its effect is durable, make handlers idempotent, and use expiring leases so another worker can recover a crashed job. Bound retries and route repeatedly failing jobs to a dead-letter queue.

Practice focus. Handle duplicate delivery, worker crashes, retries and messages that can never succeed.

Read the full worked answer

Design a Video Processing Pipeline

Answer outline. Upload video bytes directly and resumably to object storage, while a durable state machine tracks each video and transcode job. Workers claim jobs with expiring, fenced leases and write attempt-specific outputs, so a crash can trigger a retry without letting stale work overwrite the current result.

Practice focus. Separate upload, durable job state and transcoding so each can recover from failures independently.

Read the full worked answer

Design a Dropbox-Style File Sync Service

Answer outline. Split files into verified immutable blocks and publish versions through conditional metadata manifests. Let devices recover from an ordered change journal using durable cursors; preserve concurrent offline edits as conflict copies, authorize block reads against current permissions, and garbage-collect unreferenced blocks only after a safety delay.

Practice focus. Keep large uploads resumable and file versions complete while devices reconnect, conflict, lose access and restore deleted data.

Read the full worked answer

Design a Product Analytics System

Answer outline. Append validated events to a durable stream before acknowledging them, then batch them into a column store for queries by team, event and time. Stable event IDs make retries safe; an identity override table handles anonymous-to-known user merges without rewriting the event history.

Practice focus. Keep ingestion durable and deduplicated while making time-range analytics fast for each customer.

Read the full worked answer

Design a Notification System

Answer outline. Write product events through an outbox, then create notifications with stable deduplication keys. Delivery workers claim each notification through a state machine, recheck current preferences before sending, and isolate critical traffic from bulk traffic; retries must account for providers that may accept a message before timing out.

Practice focus. Prevent duplicate sends, honor preference changes and protect urgent messages when a provider slows down.

Read the full worked answer

Design a Distributed Cache (Memcache)

Answer outline. Keep the database authoritative and treat the cache as a disposable, demand-filled copy. Record invalidations with each database commit and deliver them after replication; use leases to stop an old read from restoring stale data, and a spare pool to absorb a cache-node failure.

Practice focus. Control stale reads and database load during writes, hot-key bursts and cache failures.

Read the full worked answer

Design Discord's Message Storage

Answer outline. Partition messages by channel and time bucket, then order them with time-sortable IDs so recent pages and history reads touch a small range. A routing layer keeps each channel’s reads local and coalesces identical requests; tombstones, repair and dual writes make deletes and live database moves explicit.

Practice focus. Make channel history cheap to read while keeping hot channels and database migrations manageable.

Read the full worked answer

Design a Collaborative Editor (Google Docs)

Answer outline. Give each document one fenced owner to assign durable sequence numbers, while clients apply edits locally and resend unacknowledged operations after reconnecting. Use an operation merge algorithm so concurrent edits converge; keep transient presence separate from the durable edit log.

Practice focus. Preserve one document history through concurrent edits, reconnects, owner changes and fan-out.

Read the full worked answer

Design a Reactive Database (Live Queries)

Answer outline. Run each query at a versioned snapshot and record the index ranges it reads. When a commit changes one of those ranges, rerun only the affected queries and push results with the commit timestamp; clients apply updates only when their subscriptions agree on that timestamp.

Practice focus. Update subscribed query results without polling or showing data from mixed snapshots.

Read the full worked answer

Shard a Live Database Without Downtime

Answer outline. Shard by workspace because most reads and joins stay within one workspace, and map a fixed set of logical shards to database hosts. For migration, log writes, backfill versioned rows, catch up, verify with comparisons and dark reads, then cut over reversibly; handle cross-workspace features as a separate design problem.

Practice focus. Scale write volume without rehashing every row, and migrate live data without losing newer writes.

Read the full worked answer

15 fundamentals, with direct answers

Use these concise explanations to review common trade-offs, failure handling, and guarantees. Each link opens a deeper SysGeeks guide.

What should you clarify first in a system design interview?

Answer. Identify the users and their core actions, then agree on scale, latency, availability, consistency and durability needs. Prioritize the requirements that change the architecture before drawing components.

Read more: Interview approach

How do you estimate capacity for a system design?

Answer. Estimate peak requests per second, data written per day, retention and concurrent work from the requirements. Use order-of-magnitude numbers to find likely bottlenecks, then size the parts that need different scaling strategies.

Read more: Capacity estimator

What is the difference between replication and sharding?

Answer. Replication keeps copies of the same data on multiple machines, which can improve read capacity and resilience. Sharding splits data across machines to spread storage and write load, but makes cross-shard operations harder.

Read more: Replication, Partitioning

When should you add a cache?

Answer. Cache data when repeated reads are costly and the hit rate makes the cache faster than recomputing or querying the source. Decide how stale a value may be and how writes invalidate it; caching adds consistency and failure cases.

Read more: Caching

When does a message queue help?

Answer. A queue decouples a producer from slower consumers and can absorb short bursts, giving workers time to process jobs asynchronously. It does not fix sustained overload: bound the queue, define retry and rejection behavior, and make handlers safe to retry.

Read more: Message queues

What does idempotency mean in system design?

Answer. An idempotent operation has the same effect when the same intent is repeated. Give each intent a stable key, claim it atomically, and reuse it across retries so a timeout does not trigger a second charge or job.

Read more: Idempotency

What is the difference between at-least-once and exactly-once delivery?

Answer. At-least-once delivery retries until acknowledgement, so a message may arrive more than once. Networks cannot guarantee exactly-once delivery; systems make repeated processing harmless with deduplication, idempotency, or conditional state changes.

Read more: Delivery guarantees

How do you stop a stale worker from overwriting a newer result?

Answer. Give each lease claim a monotonically increasing fencing token and require writes to present the current token. An expired worker may keep running, but storage rejects its stale token after another worker takes ownership.

Read more: Leases and fencing tokens

How can a database migration avoid downtime?

Answer. Expand the schema compatibly, backfill in bounded batches, capture concurrent writes, and compare old and new data before cutover. Keep the old path available until verification passes and rollback is no longer needed.

Read more: Online data migrations

What is backpressure, and when should a system apply it?

Answer. Backpressure limits incoming work when a downstream service cannot keep up. Bound queues and slow or reject producers so latency, memory use, and retry volume do not grow without limit.

Read more: Backpressure and capacity

What problem does the transactional outbox solve?

Answer. A database update can commit while publishing its event fails. Write the business change and an outbox record in one transaction, then relay the event with retries and make consumers handle duplicates.

Read more: Transactional outbox

How do you handle a hot partition?

Answer. A single high-volume key can overload one partition even when the rest of the cluster has capacity. Split or salt hot keys, or isolate hot tenants, then account for the extra work needed to merge reads and writes.

Read more: Partitioning

How should a payment system handle a timeout with an unknown outcome?

Answer. Treat the result as unknown rather than failed. Look up the provider operation by its stable reference and reconcile its state before retrying; never create a new charge just because the response timed out.

Read more: Reconciliation, Idempotency

What is load shedding?

Answer. Load shedding rejects or defers lower-priority work during overload to preserve capacity for critical requests. Pair it with bounded queues and clear retry behavior so rejected work does not create a retry storm.

Read more: Load shedding

When should work run asynchronously?

Answer. Use asynchronous processing when the caller does not need the result immediately and a durable queue can isolate slow work or smooth bursts. Return an operation status, then design retries, ordering, idempotency, and queue limits explicitly.

Read more: Asynchronous processing

Continue practising

Practice all 30 questions and use the interactive exercises at sysgeeks.com/investigations.